

# 代码自动生成--串讲

## 目录

- 一、代码自动生成体系
  - 1.1 CodeGen的概念
    - 1.1.1 基本概念
    - 1.1.2 引入CodeGen的意义
  - 1.2 Paddle内的CodeGen体系
- 二、动态图生成体系
  - 2.1 新动态图生成体系
    - 2.1.1 整体设计
    - 2.1.2 API生成设计
    - 2.1.3 网络构建层生成设计
      - 2.1.3.1 普通算子网络构建层
      - 2.1.3.2 组合算子网络构建层
    - 2.1.4 Python-C交互层生成设计
  - 2.2 旧动态图生成体系
    - 2.2.1 整体设计
    - 2.2.2 网络构建层生成设计
    - 2.2.3 Python-C交互层生成设计
- 三、静态图op生成体系
- 四、生成机制的完善
- 五、不足和展望
- 六、工作总结
- 附录

## 一、代码自动生成体系

### 1.1 CodeGen的概念

#### 1.1.1 基本概念

代码自动生成（Code generation）是指根据指定规范或高级描述，自动生成源代码或其他软件部件的过程。代码自动生成的目标是提高生产力，减少错误，提高软件的可维护性。

代码自动生成有不同的技术，包括基于模板的代码生成、基于模型的代码生成等。代码自动生成在现代软件开发中被广泛应用，例如在神经网络、编译器、数据库等领域中都有应用。它可以大大提高软件开发的效率，同时减少人工出错的可能性。

### 1.1.2 引入CodeGen的意义

Paddle作为国内第一的深度学习框架，功能已经十分丰富，代码量十分庞大。为了实现更多的硬件接入Paddle体系，也开放给第三方的硬件平台修改Paddle主框架的权限，与此同时也增加了框架的维护成本。鉴于此，引入CodeGen这套设计模式来解决这些问题。CodeGen能够给paddle带来以下收益：

- 1. 减少框架代码量：相关代码通过简单的yaml文件和Python脚本在编译阶段(或构建阶段)自动生成，大大减少了Paddle的可见代码量；
- 2. 降低开发成本：如果要进一步往Paddle中增加新的算子，只需要配置yaml文件、添加对应的Kernel文件和Python端接口代码，即可完成添加；
- 3. 实现Op定义统一，降低维护成本：通过固定的机制来生成相关代码，规范了代码添加规则，降低框架维护升级成本。

厂内的CINN中也大量使用CodeGen的技术生成目标代码。

### 1.2Paddle内的CodeGen体系

Paddle核心训练框架中，目前主要存在的是三套CodeGen设计体系，即动态图、静态图、旧动态图，但旧动态图在完全退场后，旧动态图的CodeGen体系也预计会被完全清除。

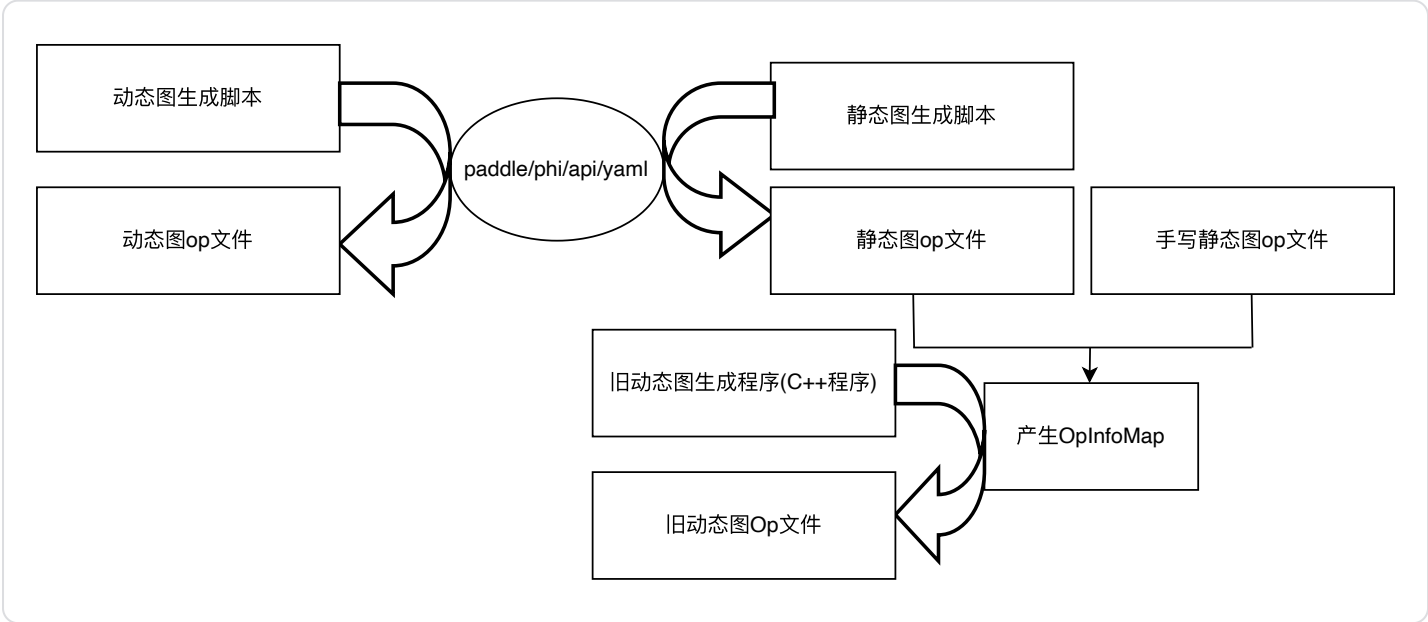


图1-1 代码自动生成总体框架

目前动态图和静态图均是依据"yaml配置文件+Python脚本"的设计自动生成相关代码，区别点在于静态图同时使用了Jinja作为生成模板。旧动态图API依据静态图注册的OpInfoMap，使用C++程序进行生成，所以旧动态图的生成的代码和静态图OP耦合度极高，这也是因为旧动态图的调用机制复用了静态图Op。这些模块内部还隐含了组合算子CodeGen、运算符重载CodeGen等模块。

首先介绍CodeGen相关的yaml配置文件，最主要的yaml文件位于paddle/phi/api/yaml/，(tensor\_operators.yaml是运算符重载yaml文件)，此外还有组合算子的一个单独yaml文件位于paddle/fluid/prim/api/api.yaml，如图1-2所示。

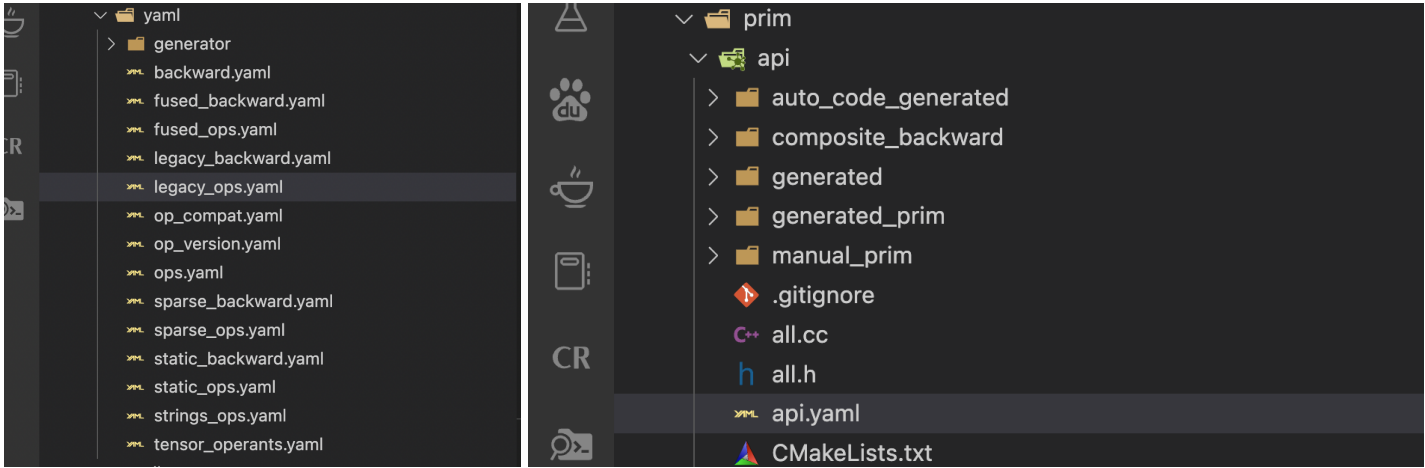


图1-2 代码自动生成相关yaml文件

相关文件yaml文件的功能如下所示：



图1-3 代码自动生成相关yaml文件功能说明

## 二、动态图生成体系

## 2.1 新动态图生成体系

### 2.1.1 整体设计

新动态图的codegen体系分为三层：**c++api**层、**dygraph\_function**网络构建层、**python-c**映射层。C++API层是纯粹的调用kernel进行计算，dygraph\_function层会根据需求建立起backward网络，将forward和backward联系起来，Python-C映射层则是方便Python端可以直接调用动态图算子接口，三者的详细介绍可以参见["动态图调用过程详解"](#)。

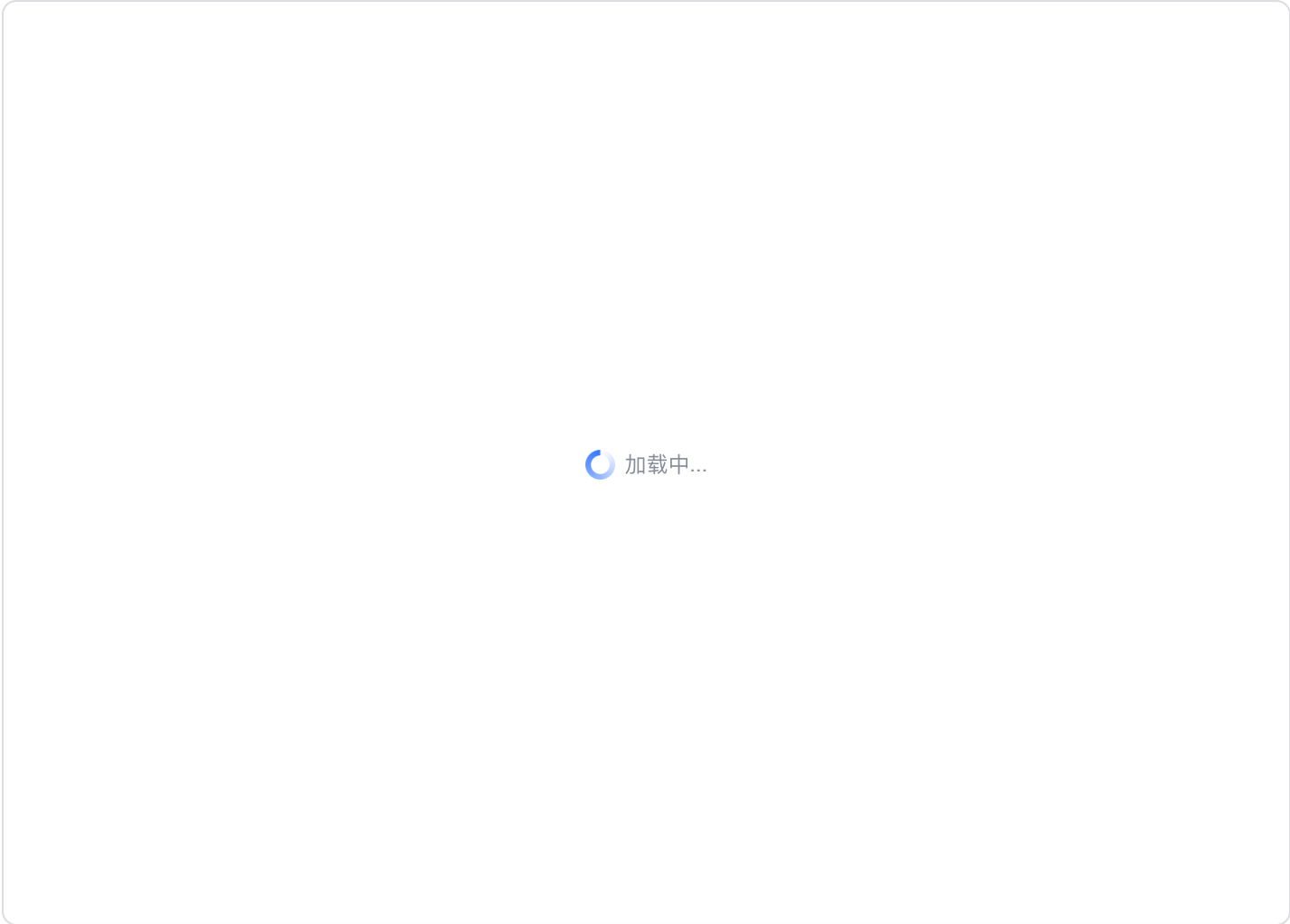


图2-1 新动态图代码自动生成框架

从图2-1可以看出，动态图三个层次的代码生成均是依据原始的yaml配置文件，只有组合算子网络构建层的API是依据parse\_op.py处理后的yaml文件。

### 2.1.2 API生成设计

API的生成脚本文件([paddle/phi/api/yaml/generator/](#))如图2-2所示：

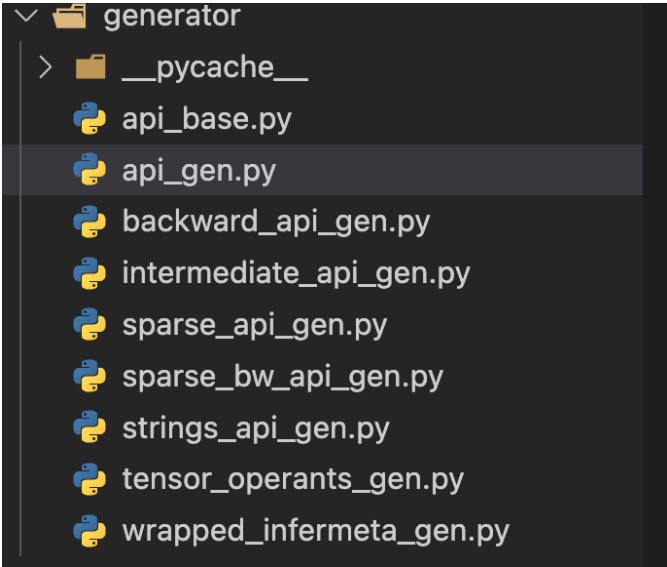


图2-2 动态图API生成脚本

每个文件的功能如下：



图2-3 API脚本功能

以api\_gen.py为例，生成代码的流程如下所示，相应的CMakeLists文件为 [paddle/phi/api/lib/CMakeLists.txt](#)：

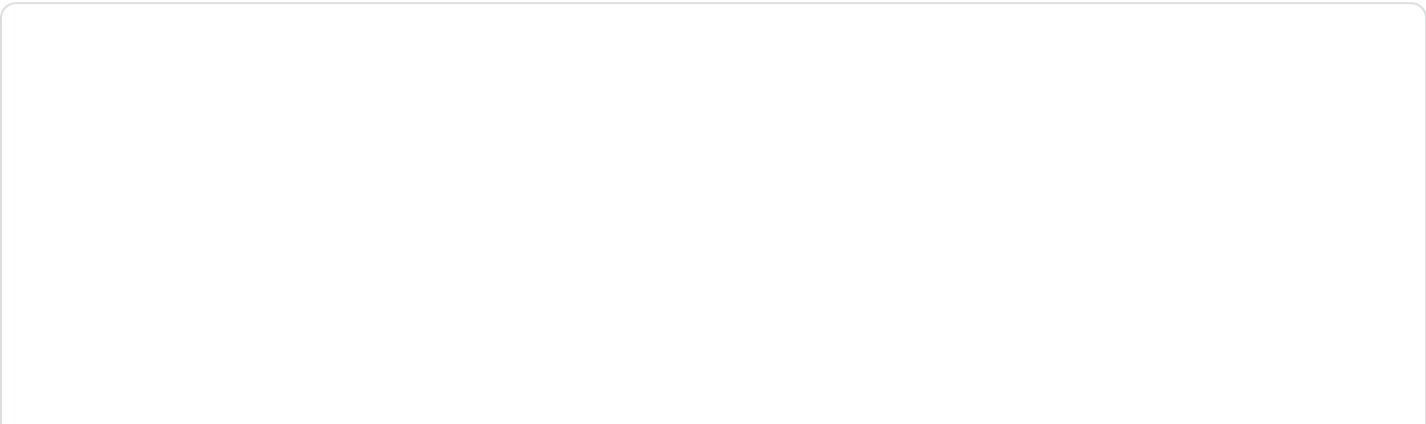


图2-4 c++API生成流程

简单的生成流程如图2-4，输入yaml文件并解析后，构造相应的生成类，调用相应的接口即可生成相应的文本，最后将文本写入文件即可。细节可以阅读相关源码。图2-4仅仅列出了普通ops也fused\_ops的生成过程，其他的sparse\_ops,infermediate\_ops,string\_ops的调用流程类似。注意Python脚本直接生成的是xxx.cc.tmp文件，最后需要通过add\_custom\_command添加编译target(output)进行拷贝生成最终的xxx.cc文件。

### 2.1.3 网络构建层生成设计

动态图网络构建层分为两个部分，一部分是基础网络构建层，一部分是组合算子网络构建层。

#### 2.1.3.1 普通算子网络构建层

普通算子网络构建层生成脚本文件(paddle/fluid/eager/auto\_code\_generator/generator/)如下：

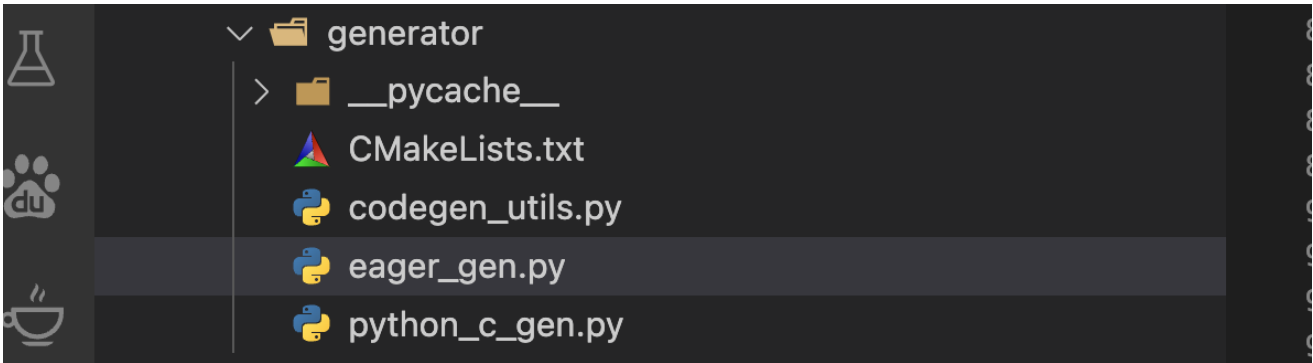


图2-5 动态图网络构建层和Python-C交互层生成脚本

由于网络构建层和Python-C交互层的脚本较为简单，二者放置在同一路径下。

图2-6 网络构建层和python-C交互层脚本功能

执行流程如下，相关CMakeLists文件  
([paddle/fluid/eager/auto\\_code\\_generator/generator/CMakeLists.txt](#)):

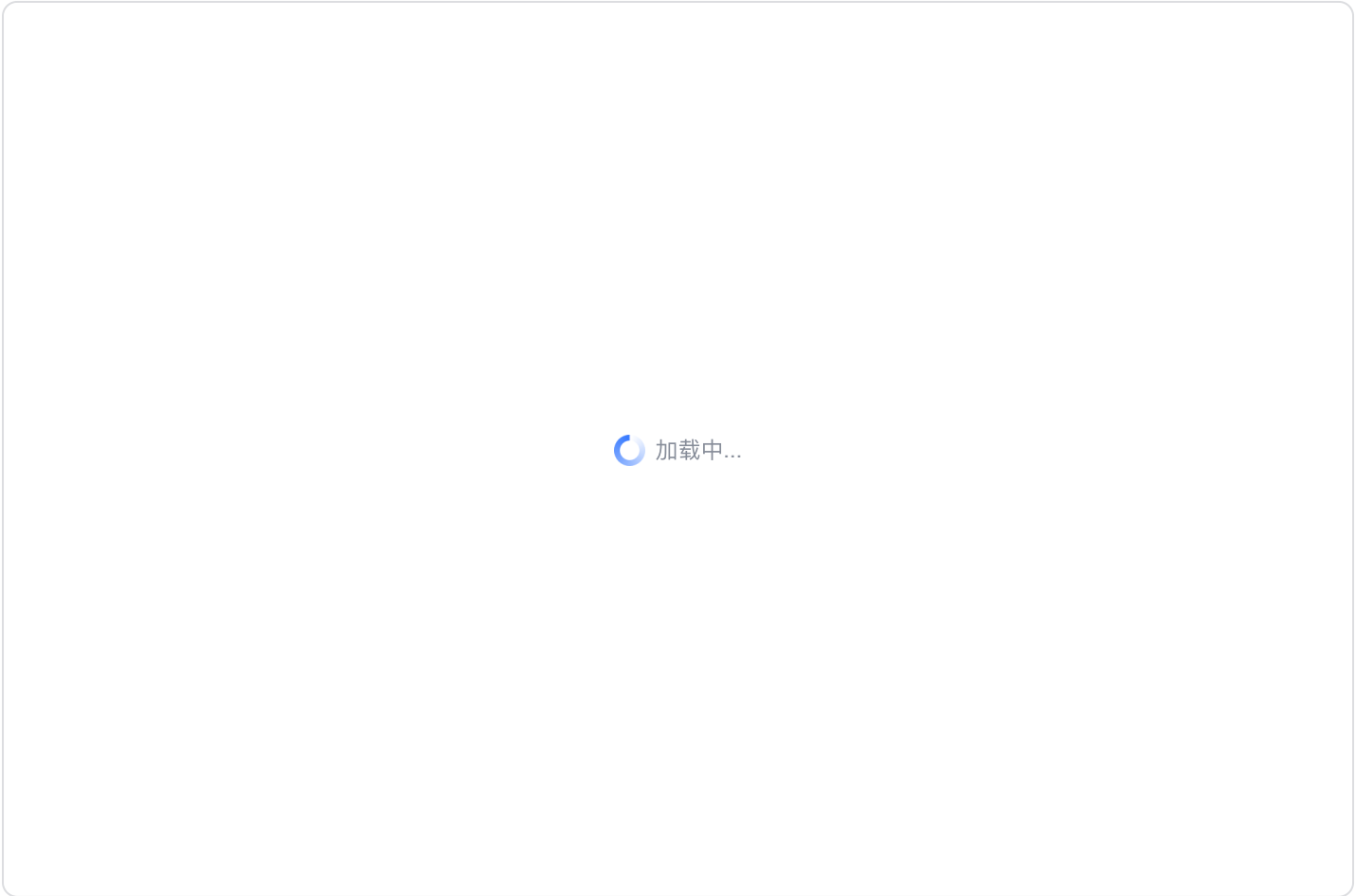


图2-7 动态图网络构建层生成流程

这里生成的文件位于[paddle/fluid/eager/api/generated/eager\\_generated](#)/路径下。  
注意Python脚本直接生成的是xxx.cc.tmp文件，最后需要通过[add\\_custom\\_command](#)添加编译target(output)进行拷贝生成最终的xxx.cc文件。

2.1.3.2 组合算子网络构建层

组合算子网络构建层生成脚本文件([paddle/fluid/prim/api/auto\\_code\\_generated](#)/)如下:

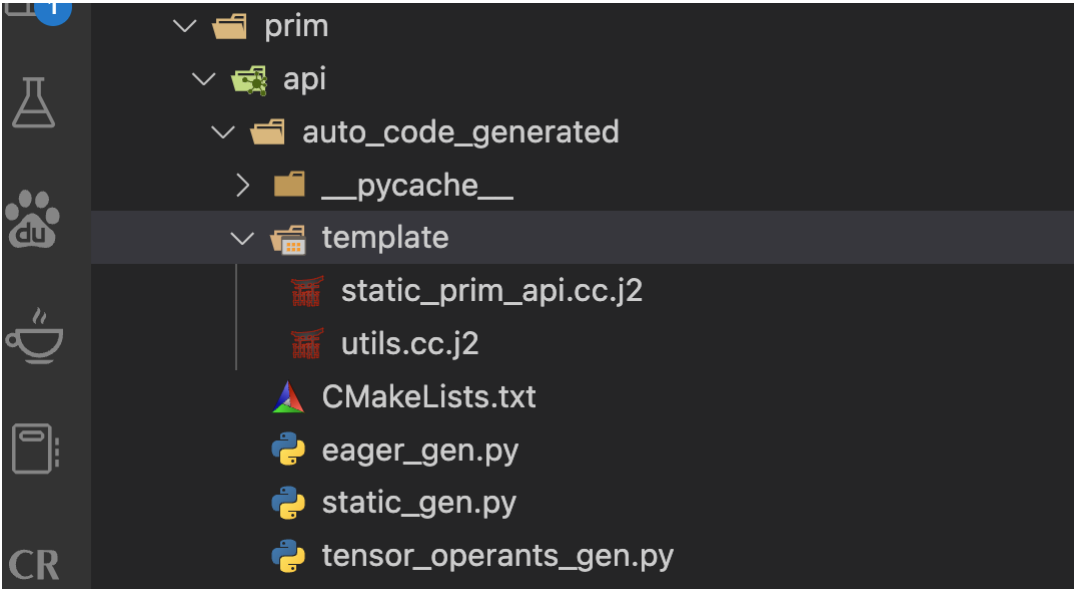


图2-8 动静态图组合算子网络构建层

组合算子网络构建层分为动态图和静态图，这里静态图代码功能类似于Python端调用append\_op这个函数。

每个文件的功能如下：



图2-9 组合算子网络构建层脚本

调用流程可以参考[paddle/fluid/prim/api/auto\\_code\\_generated/CMakeLists.txt](#)。

注意Python脚本直接生成的是xxx.cc.tmp文件，最后需要通过[execute\\_process](#)添加构建(cmake)期脚本命令进行拷贝生成最终的xxx.cc文件。

### 2.1.4 Python-C交互层生成设计

Python-C交互层相关文件和功能参见图2-5和图2-6中的[python\\_c\\_gen.py](#)文件。

从CMakeLists文件[paddle/fluid/eager/auto\\_code\\_generator/generator/CMakeLists.txt](#)中看出，通过[add\\_custom\\_target](#)添加target调用[python\\_c\\_gen.py](#)脚本直接生成最终文件。

```
</> C++ | 收起 ^  
  
1 add_custom_target(  
2   eager_python_c_codegen  
3   COMMAND
```



```

4     "${PYTHON_EXECUTABLE}"
5
6     "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/auto_code_generator/generator/python
_c_gen.py"
7     "--api_yaml_path=${api_yaml_path},${fwd_api_yaml_path}"
8     "--output_path=${tmp_python_c_output_path}"
9     COMMAND ${CMAKE_COMMAND} -E copy_if_different ${tmp_python_c_output_path}
10    ${python_c_output_path}
11    VERBATIM)

```

这里的target--eager\_python\_c\_codegen会被paddle/fluid/pybind/CMakeLists.txt下的add\_custom\_command依赖，保证add\_custom\_target会被执行：

&lt;/&gt;

C++ | 收起 ^

```

1     if(NOT ((NOT WITH_PYTHON) AND ON_INFER))
2         add_custom_command(
3             OUTPUT ${eager_impl_file}
4             COMMAND
5                 ${CMAKE_COMMAND} -E env "LD_LIBRARY_PATH=${ENV{LD_LIBRARY_PATH}}:."
6                 "${CMAKE_CURRENT_BINARY_DIR}/eager_legacy_op_function_generator"
7                 "${tmp_eager_impl_file}"
8             COMMAND ${CMAKE_COMMAND} -E copy_if_different ${tmp_eager_impl_file}
9                 ${eager_impl_file}
10            COMMENT "copy_if_different ${tmp_eager_impl_file} to
${eager_impl_file}"
11            DEPENDS ${EAGER_OP_IMPL_DEPS}
12            VERBATIM)
13    endif()

```

## 2.2 旧动态图生成体系

### 2.2.1 整体设计

由于旧动态图复用了静态图的Op(Trace模式)，所以旧动态图的代码生成的只有两部分组成，也就是网络构建层和Python-C交互层。



图2-10 旧动态图代码自动生成框架

### 2.2.2 网络构建层生成设计

旧动态图网络构建层使用的C++程序，之所以使用C++程序是方便从OpInfoMap::Instance()中获取Op的信息。文件生成器构成如下：

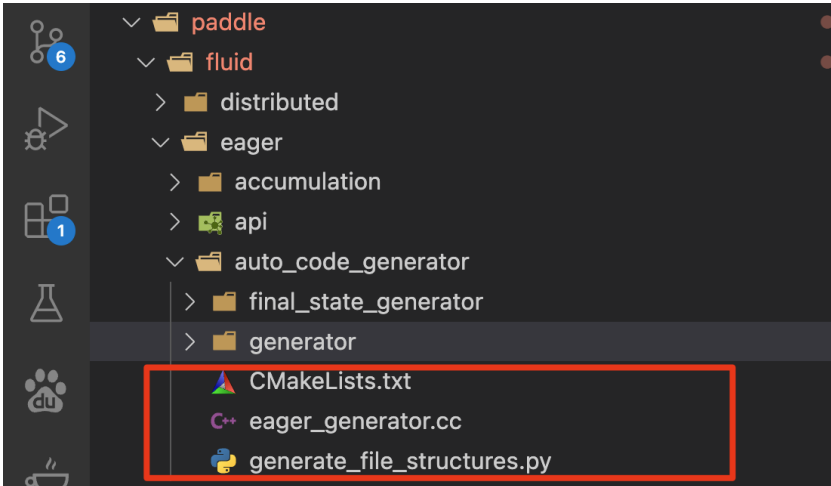


图2-11 旧动态图算子网络构建层生成器文件

相关文件功能如下：

图2-12 旧动态图网络构建层生成器

旧动态图的相关代码生成过程如下：

图2-13 旧动态图网络构建层生成流程

通过观察生成的CMakeLists.txt文件，如  
paddle/fluid/eager/api/generated/fluid\_generated/forwards/CMakeLists.txt：

```
1 add_custom_target([
2     copy_dygraph_forward_functions
3     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/dygraph
4     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
5     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
6     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
7     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
8     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
9     COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
10    COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
11    COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
12    COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
13    COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
14    COMMAND ${CMAKE_COMMAND} -E copy_if_different "${PADDLE_SOURCE_DIR}/paddle/fluid/eager/api/generated/fluid_generated/forward
15    DEPENDS legacy_eager_codegen
16    VERBATIM])
17 cc_library(dygraph_function SRCS dygraph_forward_functions1.cc dygraph_forward_functions2.cc dygraph_forward_functions3.cc dyg
18 add_dependencies(dygraph_function copy_dygraph_forward_functions copy_dygraph_node)
19
```

图2-14 相关CMakeLists.txt文件

生成lib库时只用到了带有数字后缀的相关文件，dygraph\_forward\_functions.cc并没有用到。这里分割文件的意义是防止单个文件过大，导致编译时间急剧增加。

### 2.2.3 Python-C交互层生成设计

旧动态图的Python-C交互层生成器组成较为简单仅有一个文件  
paddle/fluid/pybind/eager\_legacy\_op\_function\_generator.cc，执行流程如下：



图2-14 旧动态图Python-C生成流程

### 三、静态图op生成体系

在Python层不会通过函数形式直接调用静态图op，并且静态图组网是通过Program、Block等模块实现的，因此静态图只需要自动生成Op相关的代码。



图3-1静态图Op代码自动生成框架

如图3-1所示，yaml文件经过parse\_op.py解析后输出对应的parsed文件，静态图生成的Python脚本借助Jinja模板生成对应的op文件和sig文件。相关文件如下：

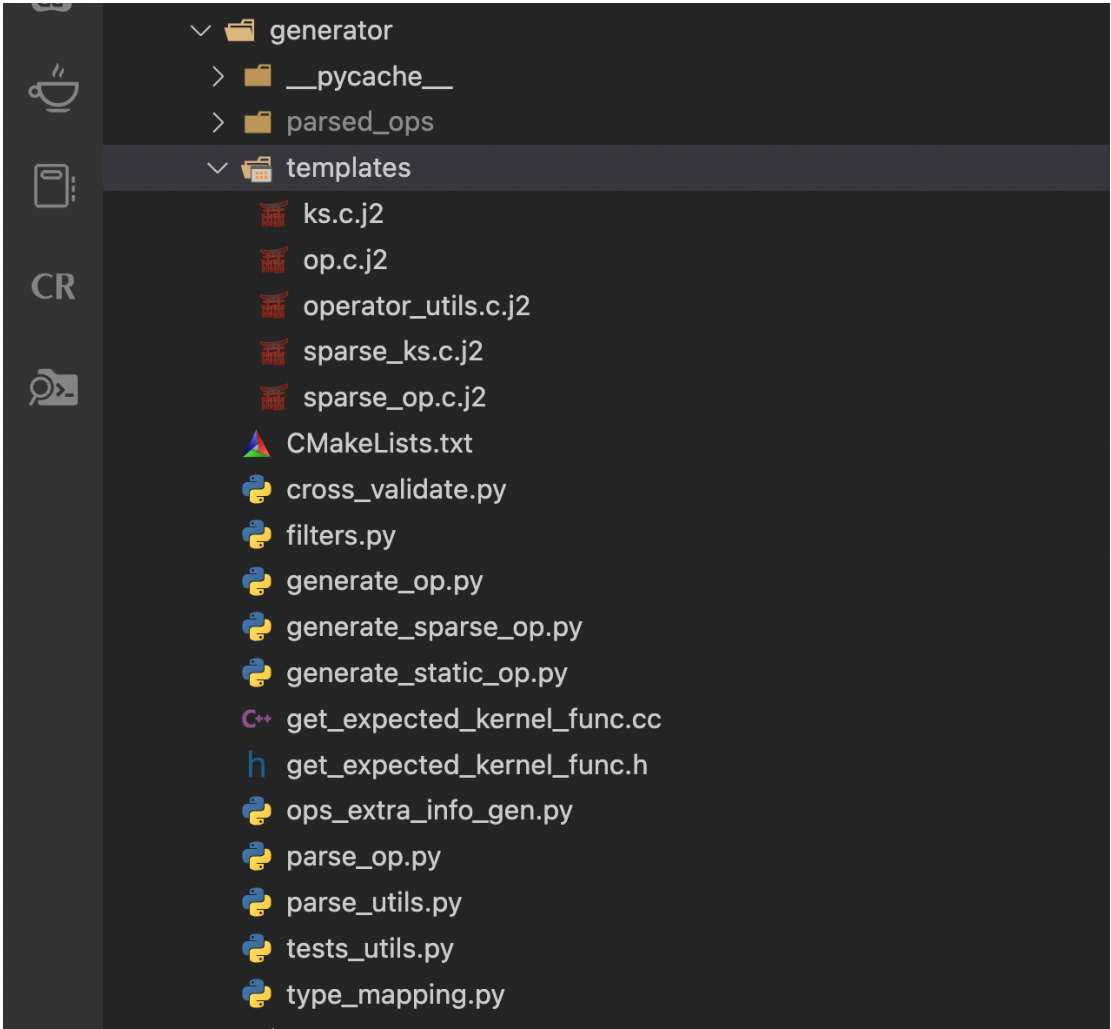


图3-2 静态图Op代码生成器相关文件

相关的文件功能如下：

图3-3 静态图Op代码生成器相关文件功能

从图中可以看出，静态图Op的生成器主要包括三部分：jinja模板，Python脚本和C++文件。静态图通过这三者之间的配合生成相关的代码文件，执行流程如下：



图3-4 静态图Op代码生成流程

从图3-4可以看到，yaml文件经过parse\_op.py解析后生成静态图规范格式的yaml文件，这一步骤的主要原始是原始的yaml配置会缺省很多配置，格式不够规整，不利于后续的jinja模板读取。为保证yaml配置的正确性，这里通过cross\_validate.py进行交叉检验，主要是通过同一个op的forward和backward的配置进行检验。经过检验正确的yaml文件会输入到generate\_op.py中，并结合op\_compat.yaml、op\_version.yaml中的信息，利用Jinja模板生成兼容手写Op文件的代码。这里的op\_compat.yaml的主要功能是参数名字映射和增加一些原始ops.yaml中没有的信息，确保生成的Op和原始手写的文件一致，新增的算子一般不需要在op\_compat.yaml中增加配置。

由于sparse相关的算子是新增到Phi下的，没有历史兼容性问题，所以不需要输入op\_compat.yaml、op\_version.yaml。ops的extra attrs的信息全部配置在op\_compat.yaml中，可以直接生成。

## 四、生成机制的完善

静态图代码的自动生成机制于去年(2022年11月)基本成型，并能够支持生成一部分符合基本设计规范的算子，还存在相当一部分算子不支持自动生成。本人的主要工作就是完善静态图的生成机制，同时在不影响兼容性的情况下清理部分算子。

|    |                     |                                                                                                      |                                                                                                                                                                                 |        |                      |
|----|---------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|----------------------|
| 1  | 问题类别                | 具体问题描述                                                                                               | 方案设计                                                                                                                                                                            | 状态     |                      |
| 2  | GetExpectedKernel   | MKLDNN问题                                                                                             | 将特殊的<br>GetExpectedKernelType函数<br>重新封装，采用invoke的方式<br>调用                                                                                                                       | 已完成    |                      |
| 3  |                     | GPUPINNED                                                                                            | 旧版本kernel支持不完善的兼容<br>形式，已删除                                                                                                                                                     | 已完成    |                      |
| 4  |                     | force_cpu参数                                                                                          | 脚本支持                                                                                                                                                                            | 已完成    |                      |
| 5  |                     | 参数X推导place,<br>dtype推导dtype                                                                          | 脚本支持                                                                                                                                                                            | 已完成    |                      |
| 6  |                     | Tensor_array类型限制<br>assign                                                                           | GetExpectedKernelType函数<br>客制化，InferVarType采用白名<br>单                                                                                                                            | 已完成    |                      |
| 7  |                     | force_cpu,<br>place_type参与选择,<br>fill_constant                                                       | invoke                                                                                                                                                                          | 依赖算子清理 | infer<br>制开<br>infer |
| 8  | GetKernelTypeForVar | MKLDNN问题                                                                                             | 针对MKLDNN污染的<br>GetKernelTypeForVar函数，将<br>这部分特殊的逻辑绑定到对应<br>的oneDNN kernel内部                                                                                                     | 已完成    |                      |
| 9  |                     | 类型提升complex                                                                                          | op_compat.yaml增加参数支持                                                                                                                                                            | 已完成    |                      |
| 10 |                     | XPU判断                                                                                                | 直接删除(测试CI可过)，潜在兼<br>容问题                                                                                                                                                         | 已完成    |                      |
| 11 | InferVarType        | 通过dtype指定out，如<br>slice、cast                                                                         | 直接删除，不能删除的加<br>infermeta中                                                                                                                                                       | 已完成    |                      |
| 12 | is_sparse参数问题       | 影响InferVarType和<br>signature                                                                         | 采用assgin方式，加入白名单                                                                                                                                                                | 已完成    |                      |
| 13 | NPU,MLU等custom宏     | NPU,MLU Make(),<br>Apply()参数设<br>置:norm,check_finite_<br>and_unscale_,cross_e<br>ntropy_with_softmax | 尝试直接删除if语句，潜在兼容<br>问题<br>1,<br>PADDLE_WITH_ASCEND_CL<br>已经完全废弃，可以直接删除<br>相关代码<br>2, PADDLE_WITH_MLU宏无<br>法直接删除，但是涉及的<br>softmax_with_cross_entropy,<br>通过查看mlu的kernel可以直接<br>删除 | 已完成    |                      |

|                 |               |                                                                                         |                               |        |                      |
|-----------------|---------------|-----------------------------------------------------------------------------------------|-------------------------------|--------|----------------------|
| 2023/9/18 13:34 |               | 代码自动生成--串讲                                                                              |                               |        |                      |
| 14              | 需要预先清理完成      | 部分XShape问题，如<br>reshape,slice                                                           | 脚本支持生成                        | 依赖算子清理 | infer<br>制元<br>infer |
| 15              |               | scalar[]参数，如<br>assign_value_                                                           | 静态图算子单独生成                     | 已完成    |                      |
| 16              | signature逻辑复杂 | IsRuntime,attr,<br>raw_kernel等                                                          | 复杂逻辑保持原始代码                    | 已完成    |                      |
| 17              | 根据atts设置参数    | Apply(),如<br>segment_pool,<br>batch_norm                                                | 对比kernel参数，直接删除if语<br>句       | 已完成    |                      |
| 18              | InferShape    | PADDLE_WITH_ASC<br>END_CL、<br>PADDLE_WITH_MLU<br>控制，如<br>cross_entropy_with_s<br>oftmax | NPU/MLU宏控制已清理                 | 已完成    |                      |
| 19              |               | 对应两个<br>InferMeta/unique                                                                | 修改signature和infermeta         | 已完成    |                      |
| 20              | 优化器           | 优化器算子动静态图配<br>置不同                                                                       | inplace参数和optional冲突，机<br>制支持 | 已完成    |                      |

表 4-1 静态图相关机制完善

表4-1列出了静态图主要缺失的机制的完善工作，这里的机制完善主要分为两部分：一是纯粹在生成机制上完善，如"complex类型提升"、"force\_cpu支持"等，二是算子清理的工作，如PADDLE\_WITH\_ASCEND\_CL、PADDLE\_WITH\_MLU宏控制等。除了表格中列出的机制完善，在清理算子的过程中还能发现机制上的一些小问题，如"解析args参数时含有"{1, 1}"类型默认值出错", "grad复用forward op并且forward含有IntArray或Scalar参数时apply参数出错的bug"等。

通过这些机制的完善目前静态图相关的算子已经在388个算子中完成了263个算子规范化列表，删除相关手写文件500多个，减少代码15000多行(按照每个文件300行估计)。并且原则上可以生成所有算子，当然存在一部分算子的InferShape无法在静态图中直接使用InferMeta函数替换，这部分算子需要进一步清理。

## 五、不足和展望

虽然目前静态图Op的生成机制已经基本完善，但依然存在以下几个问题需要进一步的处理：

1. InferShape函数需要进一步清理

虽然在yaml中配置的算子都有对应的InferMeta函数，但是由于同一个Op在动态图和静态图下的行为并不完全相同，导致在动态图下的InferMeta函数并一定能够在静态图下正确执行，尤其是含有shape



参数的算子，如reshape、slice、full(fill\_constant)算子，如何处理shape参数是需要进一步思考的问题。

## 2. 动静态图算子行为统一

部分算子的动静态图行为不统一，这些不统一的地方主要包括如下部分：

- a. 算子接口不统一：如部分动态图算子的API接口含有`Place place={}`参数，但是静态图下并没有这些参数，如arrange，导致动态图和静态图无法共用同一套yaml配置(目前这里算子配置在static\_ops.yaml中)。
- b. 一个静态图算子对应多个动态图算子API，如静态图下的matrix\_rank对应动态图下的matrix\_rank、matrix\_rank\_tol

## 3. oneDNN算子的问题

目前oneDNN kernel仅仅支持静态图，但是添加oneDNN后端kernel的时候，一些逻辑侵入到了Operator层，导致部分含有oneDNN后端的op的成员函数GetExpectedKernelType和GetKernelTypeForVar变得逻辑复杂。为解决该问题，目前静态图采用手写相关文件来支持整体的Op自动生成，但整体的规范性受到了一定程度的破坏。建议后续可以将该部分的代码进行重构。

## 4. 算子兼容性问题

目前部分算子的kernel存在冗余问题，尤其是存在大量的xxx\_raw kernel，同时静态图为了区分选择两个kernel会增加一个参数，这导致了参数的冗余。但是为考虑版本兼容问题，难以直接清理相关算子，如unique、reduce相关算子等。

## 5. 设计统一

代码生成现在已经有四套生成脚本(动态图、静态图、旧动态图、组合算子)，缺少统一的设计，这导致代码的可读性和可维护性都有问题

# 六、工作总结

加入飞桨已经接近6个月，这期间除了在做完善静态图op代码生成的工作之外也承担一些团队内部的日常工作，在这里也做一个简单的工作总结。目前静态图代码自动生成的机制基本完善，剩余的约90个算子计划通过开源社区和新人任务来完成，在清理剩余算子的过程也可能发现其他问题，尤其是旧模型的兼容性问题，后期也会参与维护。

通过这半年的工作学习，个人认为获得了以下成长：

1. 基本熟悉paddle的框架代码：尤其在静态图的组网执行流程等方面，对于静态图op的设计原理和各部分的功能已基本了解，并能够解答外部开发者和同事相关的基本问题
2. 算子相关的基本调试能力：由于清理算子并支持自动生成需要大量的调试工作，例如参数设置错误、InferMeta函数无法直接替换InferShape函数等导致自动生成的相关代码运行出错，出现这些错误，能够基于paddle的log机制进行调试解决。对于QA反馈的相关问题，也能及时解决。
3. 加强了Python以及C/C++相关知识的学习
4. 增强了解决问题的能力

# 附录

1. 通用yaml配置说明[yaml配置项目](#)
2. "快乐开源"--[框架静态图算子自动生成一期](#)
3. "快乐开源"--[框架静态图算子自动生成二期](#)
4. CMake关键命令([官方文档](#)):
  - a. `execute_process`: 直接算构建(`cmake`)期间执行脚本
  - b. `add_custom_command`: 在编译(`make`)期间执行脚本, 相当于构建一个`target`。通常有两个中用法(`OUTPUT`和绑定`TARGET`)
  - c. `add_custom_target`: 在编译(`make`)期间执行脚本, 相当于构建一个`target`, 但是默认是不会构建该`target`的, 当显示指明该`target`(`cmake .. --build --target my_target`)或这有其他`target`依赖该`target`时才会生成。
  - d. `copy_if_different`:如果是多个文件, 确保文件路径已经存在。